

Studi Kasus: Polinom Representasi Kontigu

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Deskripsi Persoalan

Sebuah polinom berderajat n didefinisikan sebagai fungsi $P(x)$ berikut:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \dots + a_2 x^2 + a_1 x^1 + a_0$$

Perhatikan bahwa untuk mempermudah pemrosesan, definisi $P(x)$ tersebut berbeda dengan definisi polinom pada matematika, yang biasanya diberikan sebagai berikut:

$$P(x) = a_0 x^n + a_1 x^{n-1} + a_2 x^{n-2} + \dots + a_{n-2} x^2 + a_{n-1} x^1 + a_n$$

Contoh Polinom:

$$P1(x) = 4x^5 + 2x^4 + 7x^2 + 10$$

$$P2(x) = 23x^{100} + 9x^9 + 2x^7 + 4x^5 + 9x^9 + 2x^4 + 3x^2 + 1$$

$$P3(x) = 10$$

$$P4(x) = 3x^2 + 2x + 8$$

$$P5(x) = x^{1000}$$

Proses Dasar Polinom (1)

1. Membentuk sebuah polinom P dari pasangan harga yang dibaca dari masukan, data yang dibaca adalah pasangan harga:

(*) $\langle \text{degree: integer, coefficient: integer} \rangle$

(1) $\langle -999, 0 \rangle$

2. Menuliskan sebuah polinom P terurut mulai dari suku terbesar sampai terkecil
3. Menjumlahkan dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$, $P3 \neq P1$ dan $P3 \neq P2$; pada akhir proses $P3 = P1 + P2$.

Proses Dasar Polinom (2)

4. Mengurangi dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$, $P3 \neq P1$ dan $P3 \neq P2$, pada akhir proses $P3 = P1 - P2$.
5. Membuat turunan dari sebuah polinom P dan menyimpan hasilnya pada P' , $P' \neq P1$, pada akhir proses P' adalah turunan $P1$.

Pemilihan Struktur Data

Secara logik sebuah suku polinom direpresentasi oleh sepasang harga integer:

< degree: integer, coefficient: integer >

$P(x)$ adalah kumpulan pasangan harga tersebut yang diidentifikasi oleh namanya dan merupakan sebuah list linier dengan elemen bertipe **Suku**, yaitu pasangan harga < degree: integer, coefficient: integer >, dengan harga **degree** yang maksimum sebagai derajat polinom.

$P(x)$ dapat direpresentasi secara **KONTIGU** atau **BERKAIT**.

Representasi Kontigu

Polinom direpresentasi dalam sebuah tabel P

- setiap elemennya berisi koefisien dari polinom
- **Derajat polinom secara implisit adalah indeks dari tabel**

KAMUS

{ Definisi sebuah polinom p adalah }

constant nMax: integer = 100

type Polinom: < degree: integer ≥ 0,
 arrSuku: array [0..nMax] of integer >

p1, p2, p3: Polinom

{ p.arrSuku[i] adalah koefisien dari suku ke-i dari sebuah polinom }

Membentuk sebuah polinom

Membentuk sebuah polinom dari pasangan harga yang dibaca dari alat masukan:

- Setiap kali memasukkan data, yang dimasukkan adalah pasangan
(*) $\langle \text{degree: integer, coefficient: integer} \rangle$
- Akhir pemasukan adalah pasangan harga yang diketikkan bernilai $\langle -999, 0 \rangle$

Prosesnya adalah **proses sekuensial dengan mark** untuk membaca dari masukan dan menyimpannya dalam tabel polinom.

Contoh: Jika dibaca P1: $\langle 0, 10 \rangle, \langle 4, -9 \rangle, \langle 5, 5 \rangle, \langle 8, 9 \rangle, \langle 2, 1 \rangle, \langle 3, 2 \rangle, \langle 9, 4 \rangle, \langle 7, 4 \rangle, \langle -999, 0 \rangle$ maka tabel polinom yang dibentuk adalah polinom berderajat 9.

Degree	TabSuku[0..100]											
9	10	0	1	2	-9	5	0	4	9	4	0 0 0	0
	0	1	2	3	4	5	6	7	8	9	...	100

Polinom “kosong”, yaitu polinom dengan Degree = -999

Menuliskan sebuah polinom

Prosesnya adalah **proses sekuensial tanpa mark**, traversal untuk i [Degree..0].

Harga suku dituliskan hanya jika koefisiennya tidak nol.

Contoh:

Degree **TabSuku[0..100]**

9	10	0	1	2	-9	5	0	4	9	4	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	...			100



I	P(I)
9	4
8	9
7	4
5	5
4	-9
3	2
2	1
0	10

Menjumlahkan dua buah polinom

Menjumlahkan dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$, $P3 \neq P1$ dan $P3 \neq P2$:

- menjumlahkan suku $P1$ dan $P2$ yang berderajat sama, menjadi suku berderajat tersebut pada $P3$
- prosesnya adalah mencari derajat tertinggi dari $P1$ dan $P2$, kemudian pemrosesan sekuensial untuk setiap pasangan suku $P1$ dan $P2$

Penjumlahan memungkinkan hasil berupa polinom kosong atau polinom yang derajatnya “turun”

Menjumlahkan dua buah polinom (2)

Contoh:

P1: $\langle 9,4 \rangle, \langle 7,4 \rangle, \langle 5,5 \rangle, \langle 4,-9 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 0,10 \rangle$

P2: $\langle 9,2 \rangle, \langle 7,3 \rangle, \langle 4,1 \rangle, \langle 1,2 \rangle$

maka P3 = P1+P2 akan mempunyai harga:

P3: $\langle 9,6 \rangle, \langle 7,7 \rangle, \langle 5,5 \rangle, \langle 4,-8 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 1,2 \rangle, \langle 0,10 \rangle$

	Degree	TabSuku[0..100]																									
P1	<table><tr><td>9</td></tr></table>	9	<table><tr><td>10</td><td>0</td><td>1</td><td>2</td><td>-9</td><td>5</td><td>0</td><td>4</td><td>0</td><td>4</td><td>0 0 0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>...</td><td>100</td></tr></table>	10	0	1	2	-9	5	0	4	0	4	0 0 0	0	0	1	2	3	4	5	6	7	8	9	...	100
9																											
10	0	1	2	-9	5	0	4	0	4	0 0 0	0																
0	1	2	3	4	5	6	7	8	9	...	100																
	+																										
	Degree	TabSuku[0..100]																									
P2	<table><tr><td>9</td></tr></table>	9	<table><tr><td>0</td><td>2</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>3</td><td>0</td><td>2</td><td>0 0 0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>...</td><td>100</td></tr></table>	0	2	0	0	1	0	0	3	0	2	0 0 0	0	0	1	2	3	4	5	6	7	8	9	...	100
9																											
0	2	0	0	1	0	0	3	0	2	0 0 0	0																
0	1	2	3	4	5	6	7	8	9	...	100																
	=																										
	Degree	TabSuku[0..100]																									
P3	<table><tr><td>9</td></tr></table>	9	<table><tr><td>10</td><td>2</td><td>1</td><td>2</td><td>-8</td><td>5</td><td>0</td><td>7</td><td>0</td><td>6</td><td>0 0 0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>...</td><td>100</td></tr></table>	10	2	1	2	-8	5	0	7	0	6	0 0 0	0	0	1	2	3	4	5	6	7	8	9	...	100
9																											
10	2	1	2	-8	5	0	7	0	6	0 0 0	0																
0	1	2	3	4	5	6	7	8	9	...	100																

Mengurangi dua buah polinom

Mengurangi dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$, $P3 \neq P1$ dan $P3 \neq P2$:

- Prosesnya sama dengan menjumlahkan, hanya operasi penjumlahan diganti operasi pengurangan.

Pengurangan juga memungkinkan hasil berupa polinom kosong atau polinom yang derajatnya “turun”.

Mengurangi dua buah polinom (2)

Contoh:

P1: $\langle 9,4 \rangle, \langle 7,4 \rangle, \langle 5,5 \rangle, \langle 4,-9 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 0,10 \rangle$

P2: $\langle 9,2 \rangle, \langle 7,3 \rangle, \langle 4,1 \rangle, \langle 1,2 \rangle$

maka P3 = P1-P2 akan mempunyai harga:

P3: $\langle 9,2 \rangle, \langle 7,1 \rangle, \langle 5,5 \rangle, \langle 4,-10 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 1,-2 \rangle, \langle 0,10 \rangle$

	Degree	TabSuku[0..100]																									
P1	<table><tr><td>9</td></tr></table>	9	<table><tr><td>10</td><td>0</td><td>1</td><td>2</td><td>-9</td><td>5</td><td>0</td><td>4</td><td>0</td><td>4</td><td>0 0 0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>...</td><td>100</td></tr></table>	10	0	1	2	-9	5	0	4	0	4	0 0 0	0	0	1	2	3	4	5	6	7	8	9	...	100
9																											
10	0	1	2	-9	5	0	4	0	4	0 0 0	0																
0	1	2	3	4	5	6	7	8	9	...	100																
—																											
	Degree	TabSuku[0..100]																									
P2	<table><tr><td>9</td></tr></table>	9	<table><tr><td>0</td><td>2</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>3</td><td>0</td><td>2</td><td>0 0 0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>...</td><td>100</td></tr></table>	0	2	0	0	1	0	0	3	0	2	0 0 0	0	0	1	2	3	4	5	6	7	8	9	...	100
9																											
0	2	0	0	1	0	0	3	0	2	0 0 0	0																
0	1	2	3	4	5	6	7	8	9	...	100																
=																											
	Degree	TabSuku[0..100]																									
P3	<table><tr><td>9</td></tr></table>	9	<table><tr><td>10</td><td>-2</td><td>1</td><td>2</td><td>-10</td><td>5</td><td>0</td><td>1</td><td>0</td><td>2</td><td>0 0 0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>...</td><td>100</td></tr></table>	10	-2	1	2	-10	5	0	1	0	2	0 0 0	0	0	1	2	3	4	5	6	7	8	9	...	100
9																											
10	-2	1	2	-10	5	0	1	0	2	0 0 0	0																
0	1	2	3	4	5	6	7	8	9	...	100																

Membuat turunan polinom

Membuat turunan P1 dari sebuah polinom P, $P1 \neq P$:

- Prosesnya adalah proses sekuensial, untuk setiap suku ke- i , $i > 0$, yaitu $a_i x^i$ pada polinom P, dihitung $i * a_i$ dan disimpan pada indeks ke- $[i-1]$ pada tabel untuk polinom P1

Membuat turunan polinom (2)

Contoh jika diberikan:

P: $\langle 9,4 \rangle, \langle 7,4 \rangle, \langle 5,5 \rangle, \langle 4,-9 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 0,10 \rangle$

maka P1 = P' akan mempunyai harga:

P1: $\langle 8,36 \rangle, \langle 6,28 \rangle, \langle 4,25 \rangle, \langle 3,-36 \rangle, \langle 2,6 \rangle, \langle 1,2 \rangle$

	Degree	TabSuku[0..100]											
P	9	10	0	1	2	-9	5	0	4	0	4	0 0 0	0
		0	1	2	3	4	5	6	7	8	9	...	100
—													
	Degree	TabSuku[0..100]											
P1	8	0	2	6	-36	25	0	28	0	36	0	0 0 0	0
		0	1	2	3	4	5	6	7	8	9	...	100

Program POLINOMIAL

{ Representasi KONTIGU }

KAMUS

```
{ Struktur data untuk representasi polinom secara kontigu}
  constant nMax: integer = 100 { Derajat tertinggi polinom yang diproses }
  type Polinom: { degree: integer,
                  arrSuku: array [0..nMax] of integer }
  p1,p2: Polinom {Operan}
  p3: Polinom { Hasil }
{ Struktur data untuk interaksi }
  finish: boolean { Mengakhiri proses }
  pilihan: integer [0..5] { Nomor tawaran }
{ Primitif operasi terhadap polinom yang dibutuhkan untuk proses }
  procedure CreatePolinom (output p: Polinom)
  { Membuat polinom p yang kosong }
  procedure adjustDegree (input/output p: polinom)
  { Melakukan adjustment terhadap Degree. Diaktifkan jika akibat suatu
    operasi, derajat polinom hasil berubah.
  }
{ Primitif operasi terhadap polinom yang disediakan untuk pemakai }
  procedure populatePol (output p: polinom) { Mengisi polinom p }
  procedure displayPol (input p: polinom)   { Menulis polinom p }
  procedure addPol (input p1, p2: polinom, output p3: polinom)
  { Menjumlahkan  $p1 + p2$  dan menyimpan hasilnya di p3,  $p3 \neq p1$  dan  $p3 \neq p2$  }
  procedure subPol (input p1, p2: polinom, output p3: polinom)
  { Mengurangkan  $p1 - p2$  dan menyimpan hasilnya di p3,  $p3 \neq p1$  dan  $p3 \neq p2$  }
  procedure derivPol (input p: polinom, output p1: polinom)
  { Membuat turunan p dan menyimpan hasilnya di p1,  $p1 \neq p$  }
```

ALGORITMA

finish $\leftarrow$ false

repeat

iterate

output ("Ketik nomor di bawah [0..5] untuk memilih operasi")

output ("1. Membentuk dua buah polinom P1 dan P2")

output ("2. Menuliskan polinom P1,P2 dan P3")

output ("3. Menjumlahkan polinom P1 dan P2 menjadi P3")

output ("4. Mengurangkan P1 dan P2 menjadi P3")

output ("5. Membentuk turunan polinom P1 yaitu P3")

output ("0. Akhir proses")

input (pilihan)

stop pilihan $\in$ [0..5]

output ("Ulangi, pilihan di luar harga yang ditawarkan")

{ Pilihan sesuai, maka lakukan proses sesuai dengan Pilihan }

depend on pilihan

 pilihan = 1: populatePol(p1); populatePol(p2)

 pilihan = 2: displayPol(p1); displayPol(p2); displayPol(p3)

 pilihan = 3: addPol(p1,p2,p3)

 pilihan = 4: subPol(p1,p2,p3)

 pilihan = 5: derivPol(p1,p3)

 pilihan = 0: finish $\leftarrow$ true

until finish

Studi Kasus: Polinom Representasi Berkait

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Deskripsi Persoalan

Sebuah polinom berderajat n didefinisikan sebagai fungsi $P(x)$ berikut:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \dots + a_2 x^2 + a_1 x^1 + a_0$$

Perhatikan bahwa untuk mempermudah pemrosesan, definisi $P(x)$ tersebut berbeda dengan definisi polinom pada matematik, yang biasanya diberikan sebagai berikut:

$$P(x) = a_0 x^n + a_1 x^{n-1} + a_2 x^{n-2} + \dots + a_{n-2} x^2 + a_{n-1} x^1 + a_n$$

Contoh Polinom:

$$P1(x) = 4x^5 + 2x^4 + 7x^2 + 10$$

$$P2(x) = 23x^{100} + 9x^9 + 2x^7 + 4x^5 + 9x^9 + 2x^4 + 3x^2 + 1$$

$$P3(x) = 10$$

$$P4(x) = 3x^2 + 2x + 8$$

$$P5(x) = x^{1000}$$

Proses Dasar Polinom (1)

1. Membentuk sebuah polinom P dari pasangan harga yang dibaca dari masukan, data yang dibaca adalah pasangan harga:

(*) $\langle \text{Degree: integer, Coefficient: integer} \rangle$

(1) $\langle -999, 0 \rangle$

2. Menuliskan sebuah polinom P terurut mulai dari suku terbesar sampai terkecil
3. Menjumlahkan dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$,
 $P3 \neq P1$ dan $P3 \neq P2$; pada akhir proses $P3 = P1 + P2$.

Proses Dasar Polinom (2)

4. Mengurangi dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$,
 $P3 \neq P1$ dan $P3 \neq P2$, pada akhir proses $P3 = P1 - P2$.
5. Membuat turunan dari sebuah polinom P dan menyimpan hasilnya pada P' ,
 $P' \neq P1$, pada akhir proses P' adalah turunan $P1$.

Pemilihan Struktur Data

Secara logik sebuah suku polinom direpresentasi oleh sepasang harga integer:

$\langle \text{Degree: integer, Coefficient: integer} \rangle$

$P(x)$ adalah kumpulan pasangan harga tersebut yang diidentifikasi oleh namanya dan merupakan sebuah list linier dengan elemen Suku, yaitu pasangan harga $\langle \text{Degree: integer, Coefficient: integer} \rangle$, dengan harga Degree yang maksimum sebagai derajat polinom.

$P(x)$ dapat direpresentasi secara **KONTIGU** atau **BERKAIT**.

Representasi Berkait (1)

- Hanya suku yang muncul saja yang disimpan datanya.
- Degree setiap suku harus disimpan secara eksplisit.
- Operasi akan lebih efisien jika suku yang muncul diurut mulai dari derajat tertinggi sampai derajat terendah

Maka:

- terjadi penghematan memori kalau suku-suku $[0..N]$ banyak yang tidak muncul.
Kalau banyak yang muncul?
- polinom kosong menjadi sangat “natural”

Representasi Berkait (2)

KAMUS

{ Definisi sebuah polinom P adalah }

type Address: ... { terdefinisi alamat sebuah suku }

type Polinom: Address { alamat elemen pertama list }

type Suku: < degree: integer,
 coef: integer,
 next: Address >

{ Polinom adalah List Suku, dengan elemen terurut menurun menurut Degree }

p1, p2, p3: Polinom

Membentuk sebuah polinom

Membentuk sebuah polinom dari pasangan harga yang dibaca dari alat masukan:

Setiap kali memasukkan data, yang dimasukkan adalah pasangan

(*) $\langle \text{Degree: integer, Coefficient: integer} \rangle$

Akhir pemasukan adalah pasangan harga yang diketikkan bernilai $\langle -999, 0 \rangle$

Proses pembentukan polinom adalah proses sekuensial untuk membaca dari masukan dan melakukan penyisipan dalam list Suku polinom yang selalu terurut menurun menurut Degree.

Membentuk sebuah polinom

Contoh: Jika dibaca P1:

$\langle 1,4 \rangle, \langle 2,5 \rangle, \langle 5,7 \rangle, \langle 8,9 \rangle, \langle 3,4 \rangle, \langle -999,0 \rangle$

maka list polinom yang dibentuk adalah polinom berderajat 8 dengan urutan penyisipan:

$\langle 1,4 \rangle$

$\langle 2,5 \rangle, \langle 1,4 \rangle$

$\langle 5,7 \rangle, \langle 2,5 \rangle, \langle 1,4 \rangle$

$\langle 8,9 \rangle, \langle 5,7 \rangle, \langle 2,5 \rangle, \langle 1,4 \rangle$

$\langle 8,9 \rangle, \langle 5,7 \rangle, \langle 3,4 \rangle, \langle 2,5 \rangle, \langle 1,4 \rangle$

Polinom “kosong”, yaitu polinom $(P) = \text{nil}$

Menuliskan sebuah polinom

Prosesnya menggunakan skema traversal dasar terhadap sebuah list polinom.

Harga suku yang dituliskan otomatis hanya yang koefisiennya tidak nol.

Contoh:

I	P(I)
8	9
5	7
3	4
2	5
1	4

Menjumlahkan dua buah polinom (1)

Menjumlahkan dua buah polinom $P1$ dan $P2$ dan menyimpan hasilnya pada $P3$

Prosesnya adalah "merging" dua buah list linier yang terurut dan setiap suku $P1$ dan $P2$

Analisis kasus:

- derajat $P1$ sama dengan derajat $P2$, jumlahkan dan sisipkan pada $P3$ (jika hasil penjumlahan tidak 0), maju ke suku $P1$ dan $P2$ yang berikutnya
- derajat $P1 >$ derajat $P2$: sisipkan suku $P1$ pada $P3$, maju ke suku $P1$ yang berikutnya
- derajat $P1 <$ derajat $P2$: sisipkan suku $P2$ pada $P3$, maju ke suku $P2$ yang berikutnya

Menjumlahkan dua buah polinom (2)

Contoh, jika diberikan:

P1: $\langle 9,4 \rangle, \langle 7,4 \rangle, \langle 5,5 \rangle, \langle 4,-9 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 0,10 \rangle$

P2: $\langle 9,2 \rangle, \langle 7,3 \rangle, \langle 4,1 \rangle, \langle 1,2 \rangle$

Maka urutan pembentukan P3 adalah:

$\langle 9,6 \rangle$ InsertLast (P3,P1+P2), Next(P1), Next(P2)

$\langle 7,7 \rangle$ InsertLast (P3,P1+P2), Next(P1), Next(P2)

$\langle 5,5 \rangle$ InsertLast (P3,P1), Next(P1)

$\langle 4,-8 \rangle$ InsertLast (P3,P1+P2), Next(P1), Next(P2)

$\langle 3,2 \rangle$ InsertLast (P3,P1), Next(P1)

$\langle 2,1 \rangle$ InsertLast (P3,P1), Next(P1)

$\langle 1,2 \rangle$ InsertLast (P3,P2), Next(P2)

$\langle 0,10 \rangle$ InsertLast (P3,P1), Next(P1)

Keadaan akhir P3: $\langle 9,6 \rangle, \langle 7,7 \rangle, \langle 5,5 \rangle, \langle 4,-8 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 1,2 \rangle, \langle 0,10 \rangle$

Mengurangi dua buah polinom

Sama dengan menjumlahkan, hanya operasi penjumlahan diganti operasi pengurangan. Contoh, jika diberikan:

P1: $\langle 9,4 \rangle, \langle 7,4 \rangle, \langle 5,5 \rangle, \langle 4,-9 \rangle$

P2: $\langle 9,2 \rangle, \langle 7,3 \rangle, \langle 4,1 \rangle, \langle 3,2 \rangle, \langle 2,1 \rangle, \langle 1,2 \rangle, \langle 0,10 \rangle$

maka urutan pembentukan P3 adalah sebagai berikut:

$\langle 9,2 \rangle$ InsertLast(P3,P1-P2), Next(P1), Next(P2)

$\langle 7,1 \rangle$ InsertLast(P3,P1-P2), Next(P1), Next(P2)

$\langle 5,5 \rangle$ InsertLast(P3,P1), Next(P1)

$\langle 4,-10 \rangle$ InsertLast(P3,P1-P2), Next(P1), Next(P2)

$\langle 3,-2 \rangle$ InsertLast(P3,P2), Next(P2)

$\langle 2,-1 \rangle$ InsertLast(P3,P2), Next(P2)

$\langle 1,-2 \rangle$ InsertLast(P3,P2), Next(P2)

$\langle 0,-10 \rangle$ InsertLast(P3,P2), Next(P2)

Dan keadaan akhir P3 adalah: $\langle 9,2 \rangle, \langle 7,1 \rangle, \langle 5,5 \rangle, \langle 4,-10 \rangle, \langle 3,-2 \rangle, \langle 2,-1 \rangle, \langle 1,-2 \rangle, \langle 0,-10 \rangle$

Membuat turunan polinom

Prosesnya adalah proses sekuensial, traversal list P, untuk setiap suku ke- i , $i > 0$: yaitu $a_i x^i$ pada polinom P, dihitung $i * a_i$ dan disisipkan P1 sebagai suku ke- $(i-1)$.

Seperti pada proses penjumlahan, list P1 dibentuk dengan penyisipan elemen terakhir, dan P1 diinisialisasi sebagai list kosong.

Karena penyisipan selalu pada akhir list, maka alamat elemen terakhir list P1 selama proses berlangsung layak untuk disimpan.

Contoh: Jika diberikan:

P: $\langle 9, 4 \rangle, \langle 7, 4 \rangle, \langle 5, 5 \rangle, \langle 4, -9 \rangle, \langle 3, 2 \rangle, \langle 2, 1 \rangle, \langle 0, 10 \rangle$

maka P1 akan mempunyai harga:

$\langle 8, 36 \rangle, \langle 6, 28 \rangle, \langle 4, 25 \rangle, \langle 3, -36 \rangle, \langle 2, 6 \rangle, \langle 1, 2 \rangle$

Representasi logik berkait	Representasi fisik berkait dengan Pointer	Representasi fisik berkait dengan Tabel
KAMUS UMUM <u>type</u> Address: ... <u>type</u> Suku: < degree: <u>integer</u> , coef: <u>integer</u> , next: Address > <u>type</u> Polinom: Address p : Polinom pt: Address	KAMUS UMUM <u>type</u> Address: <u>pointer to</u> Suku <u>type</u> Suku: < degree: <u>integer</u> , coef: <u>integer</u> , next: Address > <u>type</u> Polinom: Address p : Polinom pt: Address	KAMUS UMUM <u>constant</u> NMAX: <u>integer</u> =100 <u>type</u> Address: <u>integer</u> [0..NMAX] <u>type</u> Suku: < degree: <u>integer</u> , coef: <u>integer</u> , next: Address > <u>type</u> Polinom: Address arrSuku: <u>array</u> [0..NMAX] <u>of</u> Suku firstAvail: Address p : Polinom pt: Address
AKSES: FIRST(p) NEXT(pt) DEGREE(pt) COEF(pt)	AKSES: p pt↑.next pt↑.degree pt↑.coef	AKSES: p arrSuku[pt].next arrSuku[pt].degree arrSuku[pt].coef
PRIMITIF ALOKASI/DEALOKASI: { tergantung rep. fisik }	PRIMITIF ALOKASI/DEALOKASI: { sistem, e.g., malloc/free } pt ← newSuku(...) deallocSuku(pt)	PRIMITIF ALOKASI/DEALOKASI: { Harus direalisasi } initialize(arrSuku) pt ← newSuku(...) deallocSuku(pt)

Program POLINOMIAL1

{ Representasi BERKAIT, dengan notasi LOJIK }

KAMUS

{ Struktur data untuk representasi polinom }

type Address: ... { type terdefinisi }

type Suku: < degree: integer,
 coef: integer,
 next: Address >

type Polinom: Address

constant NIL: Address = ... { untuk address tidak terdefinisi }

p1, p2: Polinom { operan }

p3: Polinom { hasil }

{ Untuk interaksi: }

finish: boolean { mengakhiri proses }

pilihan: integer [0..5] { nomor tawaran }

{ Primitif operasi polinom untuk operasi internal }

function newSuku () → Address { Alokasi sebuah suku }

procedure deallocSuku (input/output pt: Address) { Dealokasi sebuah suku }

procedure CreatePolinom (output p: polinom) { Membuat polinom kosong P }

procedure insertLast (input pt: Address, input/output p: Polinom,
 input/output last: Address) { Insert pt sesudah
 elemen terakhir p dengan address elemen terakhir
 = last }

```

{ Primitif operasi polinom yang ditawarkan ke pengguna }
  procedure populatePol (output p1: polinom) { Mengisi polinom p1 }
  procedure displayPol (input p: polinom) { Menulis polinom p }
  procedure addPol (input p1, p2: polinom, output p3: polinom)
  { Menjumlahkan  $p1 + p2$  dan menyimpan hasilnya di p3,  $p3 \neq p1$  dan
     $p3 \neq p2$  }
  procedure subPol (input p1, p2: polinom, output p3: polinom)
  { Mengurangkan  $p1 - p2$  dan menyimpan hasilnya di p3,  $p3 \neq p1$  dan
     $p3 \neq p2$  }
  procedure derivPol (input p: polinom, output p1: polinom)
  { Membuat turunan p dan menyimpan hasilnya di p1,  $p1 \neq p$  }

```

ALGORITMA

```

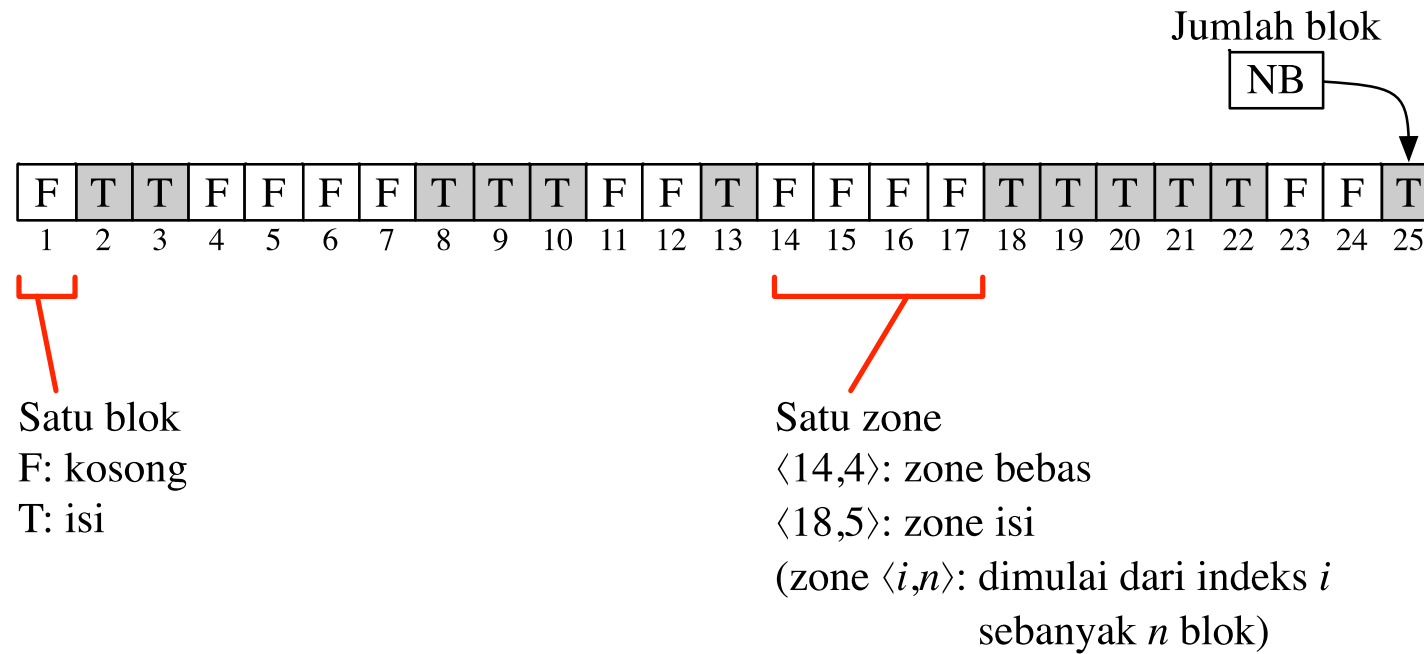
{ Sama dengan untuk representasi kontigu }

```

Studi Kasus: Pengelolaan Memori Rep. Kontigu

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Memori

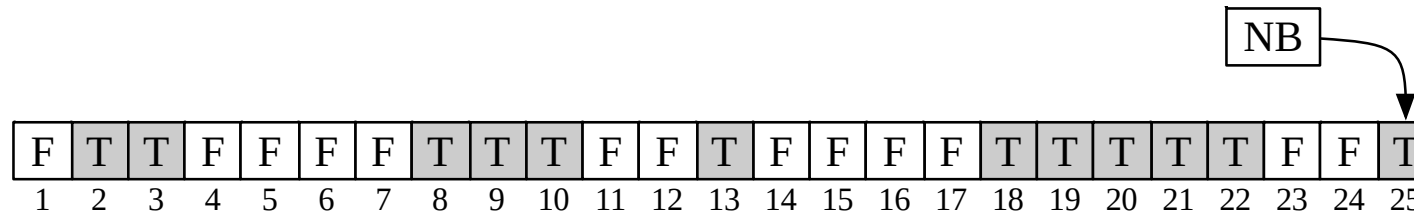


Deskripsi Persoalan

Memori dinyatakan sebagai NB buah blok kontigu

F: KOSONG

T: ISI



Zone Bebas: blok-blok berurutan yang berstatus F (KOSONG)

Dinyatakan oleh indeks blok KOSONG pertama dan ukurannya

Zone bebas I: $\langle 1,1 \rangle$

Zone bebas II: $\langle 4,4 \rangle$

Zone bebas III: $\langle 11,2 \rangle$

Zone bebas IV: $\langle 14,4 \rangle$

Zone bebas V: $\langle 23,2 \rangle$

Deskripsi Persoalan

Alokasi dan dealokasi memori menyebabkan perubahan terhadap zone bebas
Realisasikan prosedur-prosedur sebagai berikut:

Prosedur InitMem

Mengeset semua blok menjadi blok KOSONG

Prosedur AlokBlok

Melakukan alokasi: membuat blok KOSONG sejumlah X menjadi ISI

Menghasilkan indeks awal (IAw) di mana alokasi dilakukan

Prosedur DeAlokBlok

“Membebaskan” zone ISI: membuat blok ISI sejumlah X yang berawal di indeks IAw menjadi KOSONG

Prosedur GarbageCollection

Memampatkan memori sehingga semua blok KOSONG berada di bagian kiri memori dan semua blok ISI berada di kanan memori

Representasi secara kontigu dengan tabel/array

Representasi dengan Tabel/Array

Struktur Data

KAMUS

```
constant NB: integer = 100  
STATMEM: array [1..NB] of boolean  
{ tabel status memori:  
  true jika ISI, false jika kosong }
```

Representasi dengan Tabel/Array

Prosedur InitMemK

Secara umum: set seluruh elemen STATMEM menjadi bernilai false

Algoritma: Diktat hlm. 167

```
procedure InitMemK
{ I.S.: Sembarang }
{ F.S.: Semua blok memori dinyatakan KOSONG }
{ Proses: Semua status blok dengan indeks [1..NB] dijadikan
  KOSONG dengan traversal blok [1..NB].
  Proses sekuensial tanpa penanganan kasus kosong ( $NB \geq 0$ ) }
```

Representasi dengan Tabel/Array

Prosedur AlokBlok (1)

procedure AlokBlok (input X: integer, output IAw: integer)

{ I.S.: Sembarang.

X adalah banyaknya blok yang diminta untuk dialokasi,
yaitu dijadikan ISI.

IAw adalah indeks pada tabel STATMEM, yang merupakan blok
KOSONG pertama yang dialokasikan }

{ F.S.: Jika ada zone yang memenuhi syarat (ada zone dengan jumlah
blok kontigu sebanyak X atau lebih yang berstatus kosong),
maka IAw akan berisi indeks blok bebas pertama pada zone
yang dipilih untuk dialokasi;

kemudian status pemakaian memori zone tsb. dimutakhirkan.
Jika tidak ada lagi zone yang memenuhi syarat (tidak ada
zone dengan jumlah blok sebanyak X) maka IAw bernilai 0
dan status blok tetap seperti pada I.S. }

Representasi dengan Tabel/Array

Prosedur AlokBlok (2)

Strategi First Fit:

Alokasi dilakukan pada zone yang memenuhi syarat yang **pertama kali ditemukan**.

Diktat hlm. 168

```
procedure AlokBlokKF (input X: integer, output IAw: integer)
```

```
Inisialisasi
```

```
repeat
```

```
    Search blok kosong pertama: skema search
```

```
    if blok pertama zone kosong then
```

```
        Catat indeks sebagai NAwal, blok awal zone kosong
```

```
        Hitung NKosong, banyaknya blok dlm zone kosong tersebut
```

```
until semua blok diperiksa atau NKosong  $\geq$  X
```

```
Terminasi
```

```
if ada zone memenuhi syarat then
```

```
    Ubah status blok pada zone tersebut [IAw..IAw+X-1] menjadi
```

```
ISI
```

Representasi dengan Tabel/Array

Prosedur AlokBlok (3)

Strategi Best Fit:

Alokasi dilakukan terhadap zone yang memenuhi syarat dan berukuran sama dengan X, atau jika tidak ada zone berukuran X, maka diambil yang ukurannya minimal

Keuntungan: blok tidak terpartisi kecil-kecil

Diktat Hlm. 169

```
procedure AlokBlokKB (input X: integer, output IAw: integer)
  Inisialisasi
  repeat
    Search zone kosong pertama: skema search
    if blok pertama zone kosong then
      Hitung banyaknya blok dlm zone kosong tsb.
    if ada zone kosong dan memenuhi syarat then
      if zone kosong pertama then
        Inisialisasi
        Ukuran zone Minimum: NBMin dan Posisi Awal NAwal
      else Cek apakah lebih baik, jika ya, update NBMin dan IAw
  until semua blok diperiksa atau blok berukuran = X
  Terminasi
  if ada zone memenuhi syarat then
    Ubah status blok pada zone tersebut [IAw..IAw+X-1] menjadi ISI
```

Representasi dengan Tabel/Array

Prosedur DeAlokBlok

Secara umum: Pemrosesan sekuensial (traversal) untuk membuat status blok [IAw..IAw+X-1] KOSONG

Diktat hlm. 170

```
procedure DeAlokBlok (input X, IAw: integer)  
{ I.S.: X adalah ukuran zone, bilangan positif dan IAw adalah  
    alamat blok awal zone tersebut, dengan  $IAw \in [1..NB-X+1]$ ,  
    Blok dengan indeks IAw s.d. IAw+X-1 pasti berstatus ISI. }  
{ F.S.: Tabel status memori dengan indeks blok IAw..IAw+X-1  
    menjadi KOSONG }  
{ Proses: Sebuah zone berukuran X dan berawal pada blok IAw  
    di-dealokasi (statusnya dijadikan kosong) }
```

Representasi dengan Tabel/Array

Prosedur GarbageCollection (1)

procedure GarbageCollection

{ I.S.: Sembarang }

{ F.S.: Tabel status memori menjadi dua bagian:

zone bebas di belahan “kiri” (indeks kecil),

zone ISI di belahan “kanan”.

Jika K adalah integer $[0..NB]$ yang merupakan indeks blok KOSONG terakhir pada zone bebas, maka ada 3 kemungkinan F.S.:

1. Jika $K \neq 0$ dan $K < NB$, maka blok dengan indeks $[1..K]$ adalah zone KOSONG, bagian dengan indeks $[K+1..NB]$ adalah zone ISI
2. Jika $K \neq 0$ dan $K = NB$: $[1..K]$ maka semua blok adalah KOSONG, memori terdiri dari sebuah zone KOSONG.
3. Jika $K = 0$ maka semua blok adalah ISI, tidak ada zone kosong. }

Representasi dengan Tabel/Array

Prosedur GarbageCollection (2)

Proses: “menggeser” elemen tabel yang berstatus KOSONG ke kiri

Dua pass (diktat hlm. 170):

- Traversal untuk mencacah banyaknya blok kosong, misalnya NKosong

- Traversal untuk memberi status $[1..NKosong]$ dengan KOSONG dan $[NKosong+1..NB]$ dengan ISI

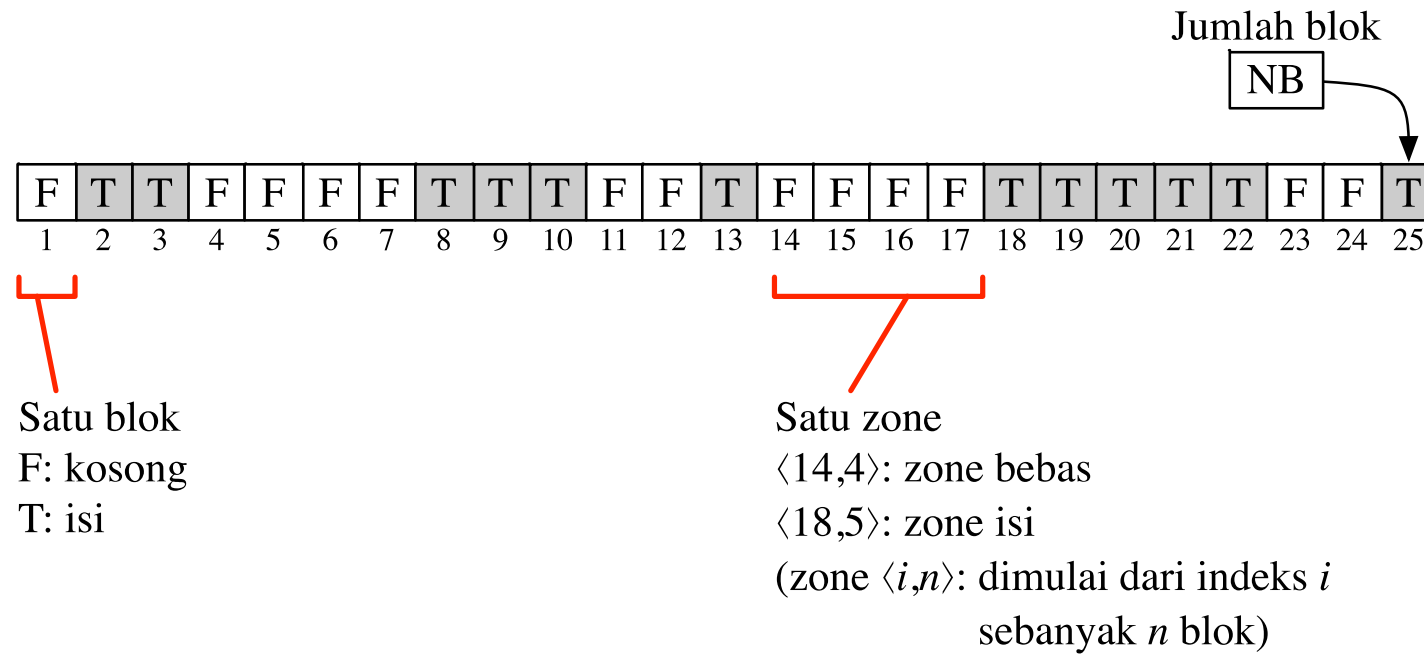
Satu pass:

- Traversal untuk mengelompokkan KOSONG di kiri dan ISI di kanan dengan menukarkan dua elemen.

Studi Kasus: Pengelolaan Memori Rep. Berkait – Blok Kosong

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Memori

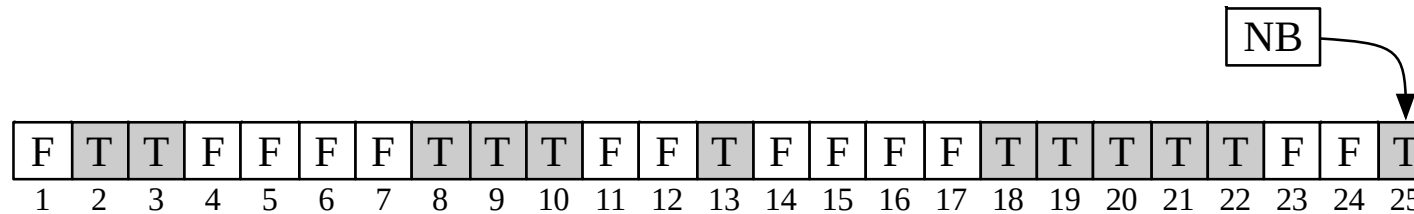


Deskripsi Persoalan

Memori dinyatakan sebagai NB buah blok kontigu

F: KOSONG

T: ISI



Zone Bebas: blok-blok berurutan yang berstatus F (KOSONG)

Dinyatakan oleh indeks blok KOSONG pertama dan ukurannya

Zone bebas I: $\langle 1,1 \rangle$

Zone bebas II: $\langle 4,4 \rangle$

Zone bebas III: $\langle 11,2 \rangle$

Zone bebas IV: $\langle 14,4 \rangle$

Zone bebas V: $\langle 23,2 \rangle$

Deskripsi Persoalan

Alokasi dan dealokasi memori menyebabkan perubahan terhadap zone bebas
Realisasikan prosedur-prosedur sebagai berikut:

Prosedur InitMem

Mengeset semua blok menjadi blok KOSONG

Prosedur AlokBlok

Melakukan alokasi: membuat blok KOSONG sejumlah X menjadi ISI

Menghasilkan indeks awal (IAw) di mana alokasi dilakukan

Prosedur DeAlokBlok

“Membebaskan” zone ISI: membuat blok ISI sejumlah X yang berawal di indeks IAw menjadi KOSONG

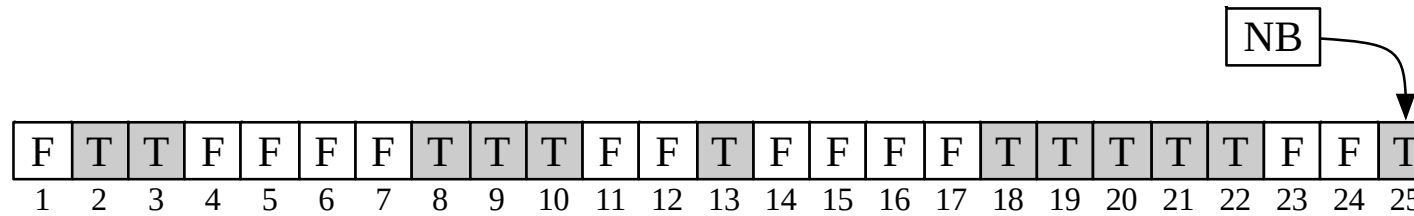
Prosedur GarbageCollection

Memampatkan memori sehingga semua blok KOSONG berada di bagian kiri memori dan semua blok ISI berada di kanan memori

Representasi Berkait Blok Kosong

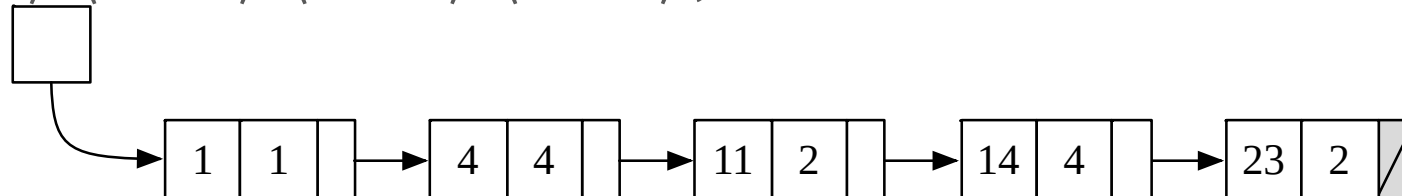
Representasi Berkait Blok Kosong

Ilustrasi



List zone kosong:

{ $\langle 1,1 \rangle$, $\langle 4,4 \rangle$, $\langle 11,2 \rangle$, $\langle 14,4 \rangle$, $\langle 23,2 \rangle$ }



Representasi Berkait Blok Kosong

Struktur Data

KAMUS

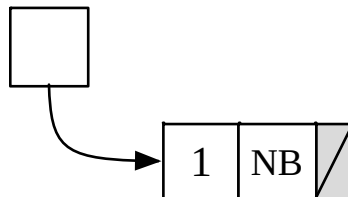
```
type: address          { type terdefinisi }
type ZB: < Aw: integer, { indeks awal zone kontigu kosong }
           Nb: integer, { banyaknya blok zone kontigu kosong }
           Next: address >
FIRSTZB: address        { address zone kosong pertama }
{ Elemen list terurut menurut Aw }
{ Fungsi akses untuk penulisan algoritma secara logik }
{ Jika P adalah sebuah address, maka dituliskan:
  - Next(P) adalah address elemen list sesudah elemen beralamat P
  - Aw(P) adalah nilai blok kosong pertama pada sebuah elemen
    list beralamat P yang mewakili sebuah zone kontigu
  - Nb(P) adalah ukuran blok sebuah zone kosong yang disimpan
    informasinya dalam elemen list beralamat P
  - Allocate(P) adalah prosedur utk melakukan alokasi sebuah ZB
    dengan alamat P, P tidak mungkin sama dengan Nil (alokasi
    selalu berhasil)
  - Deallocate(P) adalah prosedur utk mendealokasi sebuah alamat P }
```

Representasi Berkait Blok Kosong

Prosedur InitMem

Algoritma: diktat hlm. 176

```
procedure InitMemB
{ I.S.: Sembarang }
{ F.S.: Semua blok memori dinyatakan KOSONG }
{ Proses: Diinisialisasi satu zone kosong <1, NB>
        jika NB adalah jumlah maksimum blok. }
{ Solusi umum:
  1. Create list kosong
  2. Allocate sebuah elemen baru P dengan NAw=1 dan Ukuran NB
  3. Insert elemen P dalam List }
```



Representasi Berkait Blok Kosong

Prosedur AlokBlok – First Fit (1)

```
procedure AlokBlokBF (input X: integer, output IAw: integer)
{ I.S.: Sembarang. X adalah banyaknya blok yang diminta untuk
  dialokasi, yaitu dijadikan ISI }
{ F.S.: Tergantung kepada proses }
{ Proses: Sequential search sebuah elemen list dengan properti
  jumlah blok kontigunya lebih besar atau sama dengan X.
  Pencarian segera dihentikan jika diketemukan elemen
  list yang memenuhi persyaratan tersebut. }
{ Hasil pencarian menentukan F.S.:
  1. Jika ada, maka ada dua kemungkinan:
    a. jika jumlah blok kontigu sama dengan X, hapus elemen
       list kosong tersebut,
    b. jika jumlah blok kontigu lebih besar dari X,
       update elemen list kosong tersebut.
  2. Jika tidak ada elemen list yang memenuhi syarat:
     keadaan list tetap dan IAw diberi nilai 0. }
```

Representasi Berkait Blok Kosong

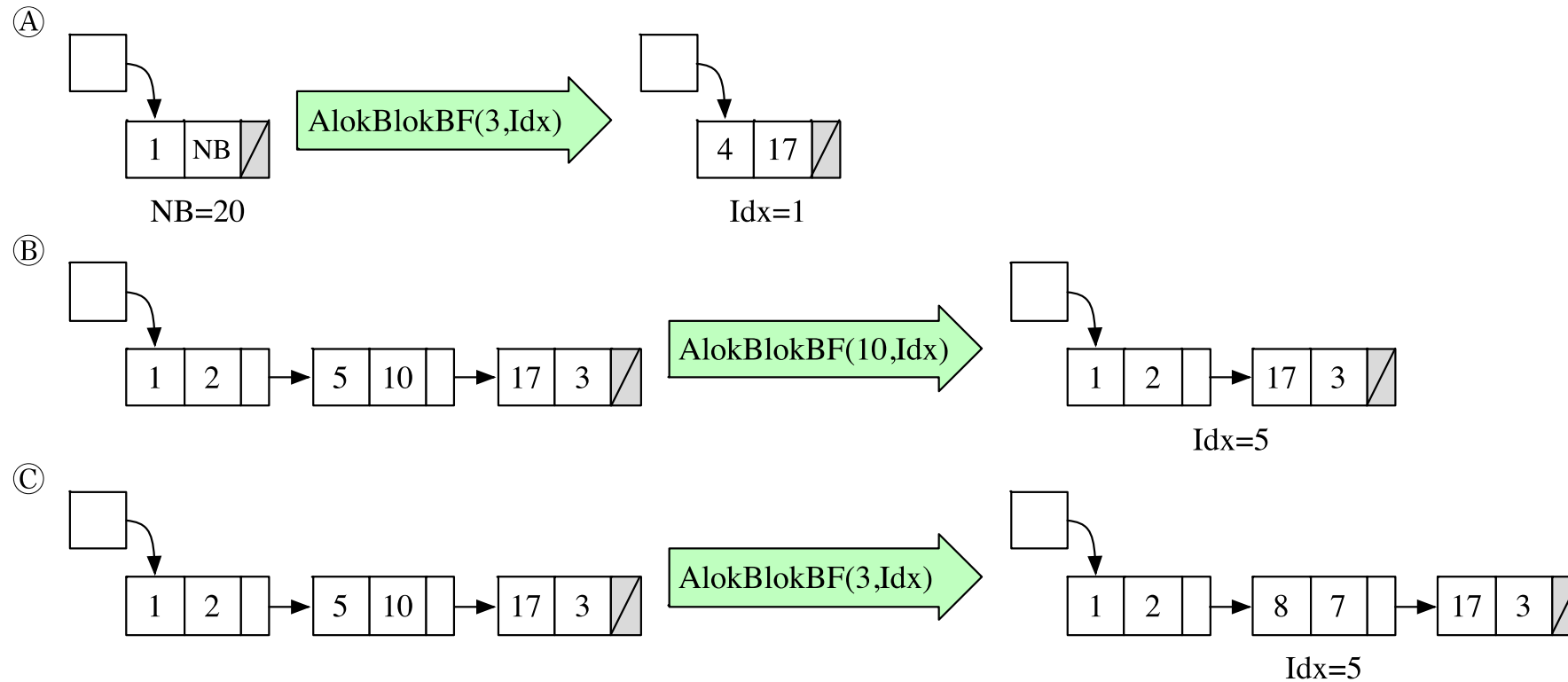
Prosedur AlokBlok – First Fit (2)

Algoritma: diktat hlm. 177

Sketsa umum algoritma:

```
Sequential search List FirstZB, P sebuah address elemen
kondisi berhenti semua elemen list diperiksa atau  $Nb(P) \geq X$ 
if ( $Nb(P) \geq X$  then      { ada yg memenuhi syarat }
    if  $Nb(P) = X$  then { zone kosong menjadi isi }
        Delete elemen beralamat P; Dealokasi P
    else { lebih besar: update zone kosong }
        Update  $Nb(P)$  dan  $Aw(P)$ 
        Set nilai  $IAw$  dengan  $Aw(P)$ 
else Set  $IAw = 0$ 
```

Ilustrasi Alokasi: First Fit



Representasi Berkait Blok Kosong

Prosedur AlokBlok – Best Fit (1)

procedure AlokBlokBB (input X: integer, output IAw: integer)

{ I.S.: Sembarang.

 X adalah banyaknya blok yang diminta untuk dialokasi,
 yaitu status memorinya dijadikan ISI }

{ F.S.: Tergantung kepada proses }

{ Proses: Periksa semua elemen list, elemen list dengan properti
 jumlah blok kontigunya lebih besar atau sama dengan X
 ditandai yang minimum. }

{ Setelah semua elemen diperiksa, ada dua kemungkinan:

1. Jika alokasi dapat dilakukan, ada blok yang memenuhi syarat,
 masih ada dua kemungkinan:
 - a. jika jumlah blok kontigu sama dengan X, hapus elemen list
 kosong tersebut,
 - b. jika jumlah blok kontigu lebih besar dari X, update elemen
 list kosong tersebut.
2. Jika alokasi tidak dapat dilakukan (tidak ada blok yang
 memenuhi syarat), maka list tetap keadaannya dan IAw diberi
 nilai 0 }

Representasi Berkait Blok Kosong

Prosedur AlokBlok – Best Fit (2)

Algoritma: diktat hlm. 178

Sequential search List FirstZB, P sebuah address elemen
kondisi berhenti $Nb(P) = X$ atau semua elemen list diperiksa
Skema search dengan boolean.

Untuk setiap elemen list beralamat P yang diperiksa:

if elemen pertama then

 Inisialisasi NBMin dan Naw

else { bukan elemen pertama }

 Cek apakah $Nb(P) < NBMIN$, jika ya update NBMin}

if ($Nb(P) \geq X$) then { ada yg memenuhi syarat }

if ($Nb(P) = X$) then { zone kosong menjadi isi }

 Delete elemen beralamat P; dealokasi P

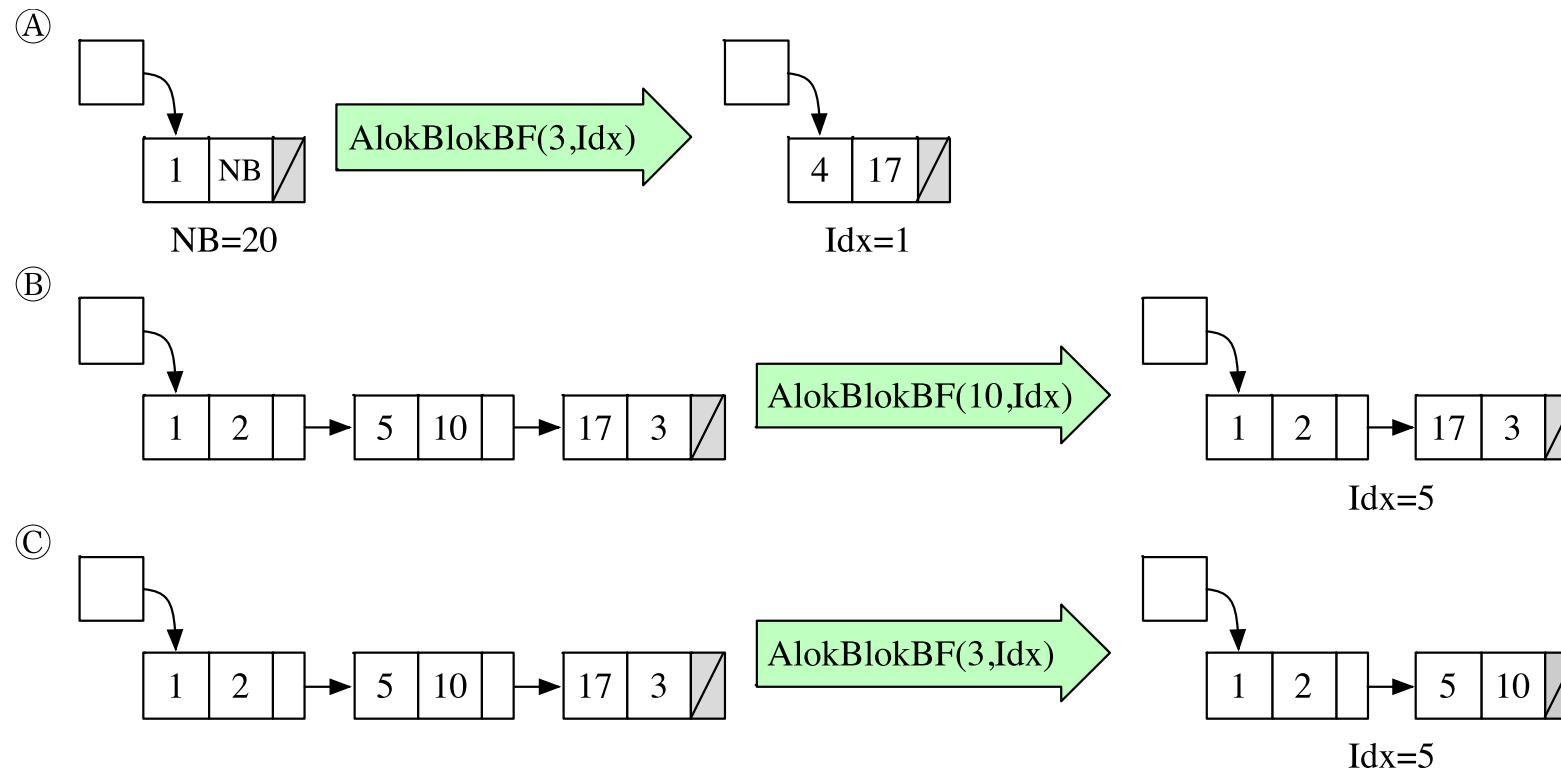
else { lebih besar: update zone kosong }

 Update $Nb(P)$ dan $Aw(P)$

 Set nilai IAw dengan $Aw(P)$

else Set IAw = 0

Ilustrasi Alokasi: Best Fit



Representasi Berkait Blok Kosong

Prosedur DeAlokBlok (1)

Algoritma: diktat hlm. 179

```
procedure DeAlokBlokB (input IAw: integer, input X: integer)  
{ I.S.: X adalah ukuran zone, bilangan positif dan IAw adalah  
      alamat blok awal zone tersebut, dengan  $IAw \in [1..NB-X]$ ,  
      blok dengan indeks IAw s.d.  $IAw+X-1$  pasti berstatus ISI. }  
{ F.S.: Tabel status memori dengan indeks blok  $IAw..IAw+X-1$   
      menjadi KOSONG. }  
{ Proses: Sebuah zone berukuran X dan berawal pada blok IAw  
      didealokasi (statusnya dijadikan KOSONG). }
```

Representasi Berkait Blok Kosong

Prosedur DeAlokBlok (2)

Kasus-kasus pada proses dealokasi:

- Zone yang dibebaskan mengubah elemen pertama list:

 - Hanya mengubah elemen pertama list

 - Insert first → menambah zone bebas di awal list

- Zone yang dibebaskan mengubah elemen terakhir list:

 - Hanya mengubah elemen terakhir list

 - Insert last → menambah zone bebas di akhir list

- Zone yang dibebaskan berada di tengah list, di antara elemen KIRI dan KANAN:

 - KIRI dan KANAN digabung → salah satu di-delete, lainnya di-update

 - Di tengah KIRI dan KANAN, tapi tidak bersambung → insert elemen baru di antara KIRI dan KANAN

 - Terletak sesudah elemen KIRI → update KIRI

 - Terletak sebelum elemen KANAN → update KANAN

Jauh lebih rumit daripada representasi secara kontigu!

Representasi Berkait Blok Kosong

Prosedur GarbageCollection

Algoritma: Diktat hlm. 180

```
procedure GarbageCollectionB
{ I.S.: Sembarang }
{ F.S.: Semua blok kosong ada di kiri dan blok isi di kanan:
    Jika ada zone kosong, hanya ada satu elemen list,
    karena dijadikan satu zone kosong.
    Jika tidak list zone kosong tidak ada elemennya (kosong),
    maka tidak dilakukan apa-apa. }
{ Proses: Jika list zone kosong tidak kosong, jadikan sebuah
    list dengan elemen tunggal beralamat P,
    Aw(P) bernilai 1 dan Nb(P) bernilai total blok kosong
    (dihitung melalui iterasi list zone kosong yang lama). }
```

Studi Kasus: Multi-List

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Deskripsi Persoalan

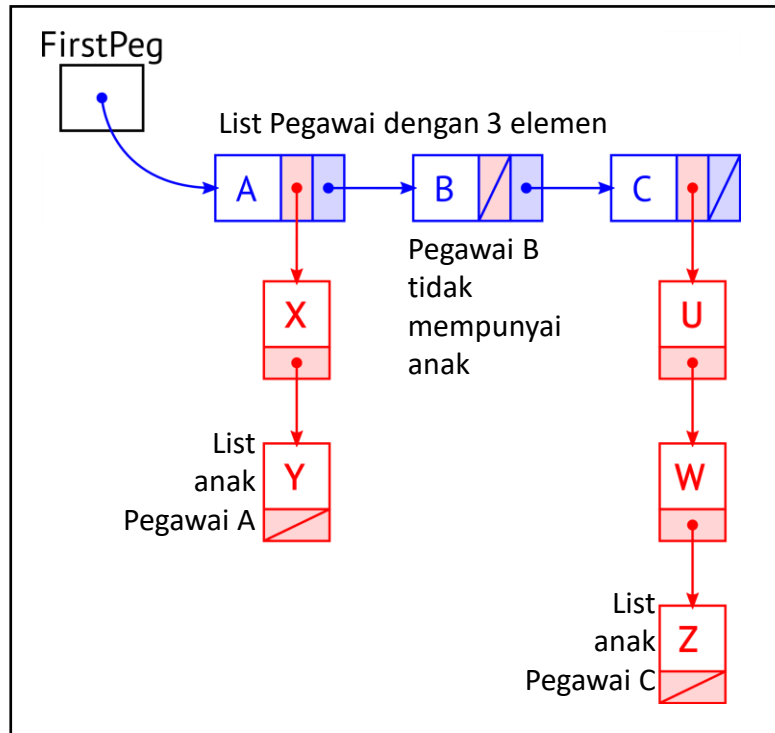
Kita harus mengelola sekumpulan pegawai, dan untuk setiap pegawai selain **informasi mengenai dirinya**, kita juga harus menyimpan **informasi tentang anak-anaknya** (jika ada).

Jika informasi tersebut harus direpresentasikan dalam struktur data internal, maka kita mempunyai list dari pegawai, dan juga list dari anak-anak pegawai.

Informasi pegawai: nopeg, nama, jabatan, gaji

Informasi anak: nama, tanggal lahir

Alternatif Struktur Data



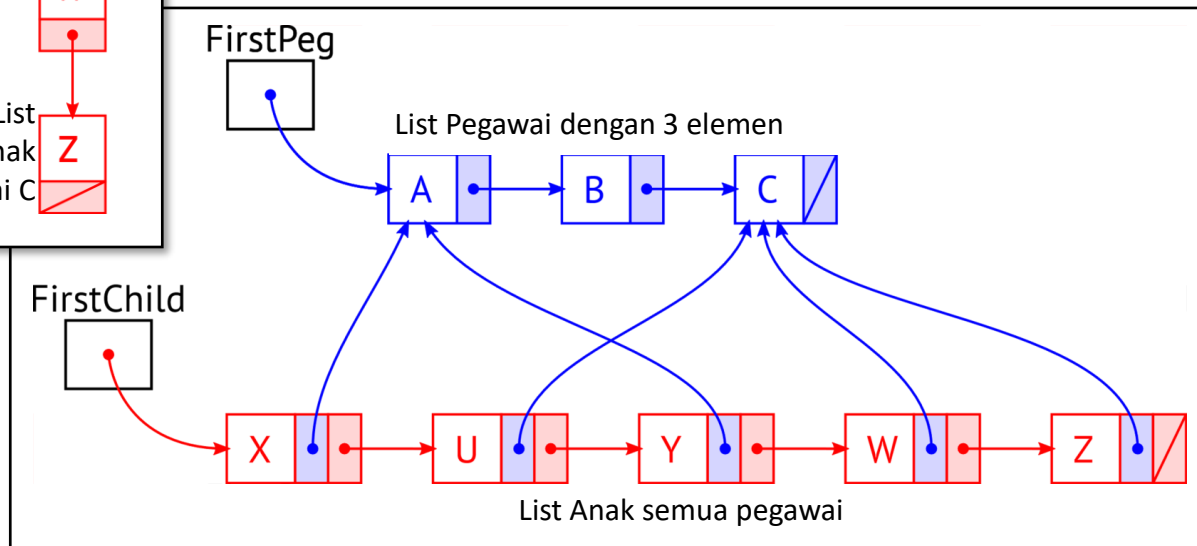
Pegawai: A, B, C, ...

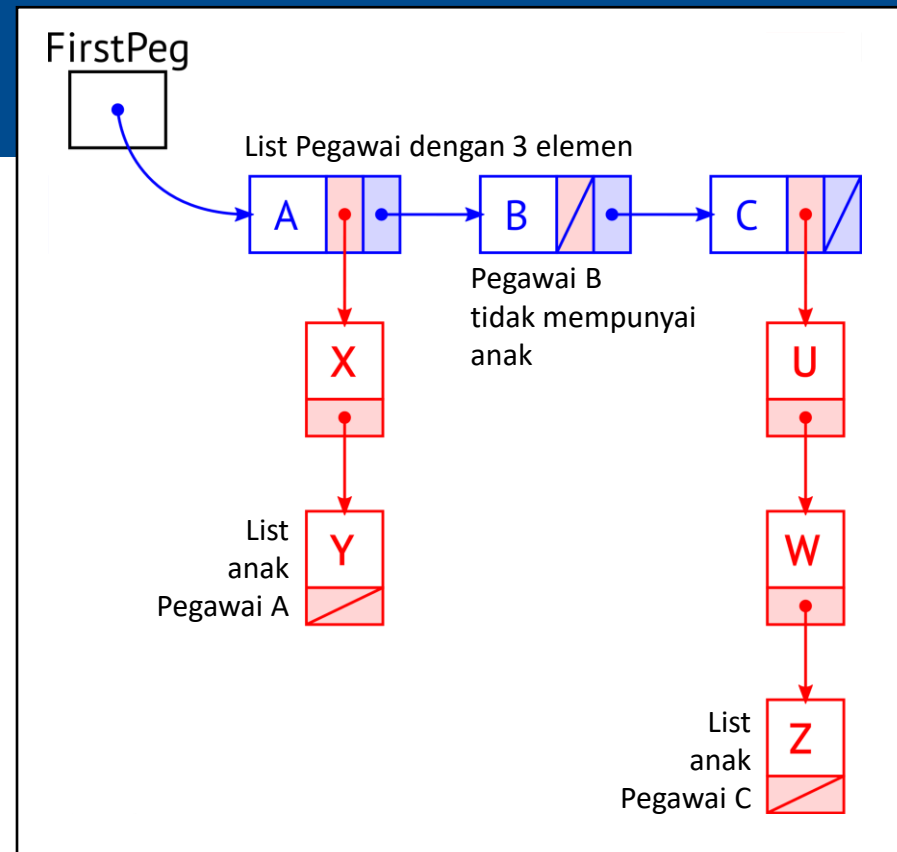
Anak: A: X,Y

B: -

C: U,W,Z

Mana yang lebih baik ?





KAMUS

type AdrPeg: ... { type terdefinisi, alamat sebuah elemen list pegawai }

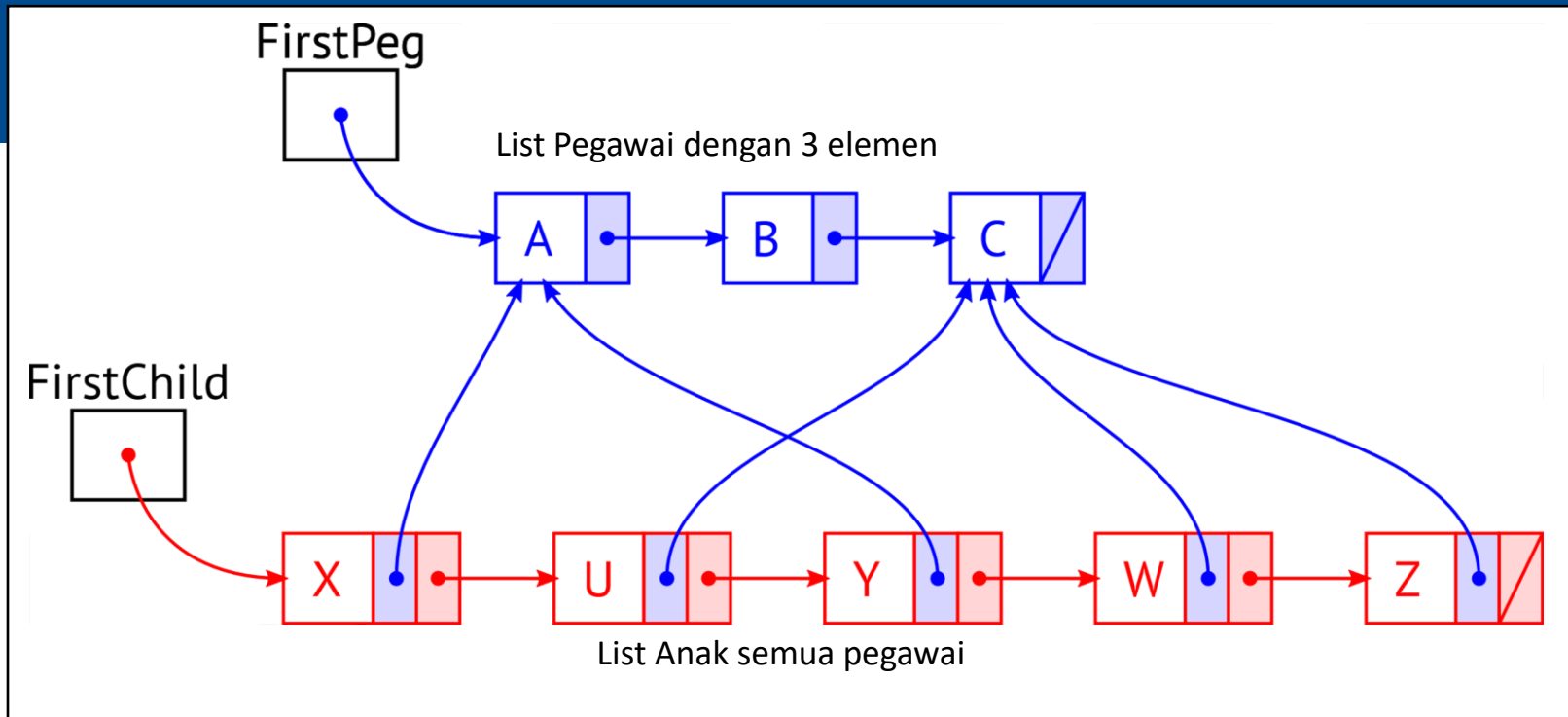
type AdrAnak: ... { type terdefinisi, alamat sebuah elemen list anak }

type Pegawai: < nip: integer, nama: string, jabatan: string,
qajiPokok: real, firstAnak: AdrAnak, nextPeg: AdrPeg >

type Anak: < nama: string, tglLahir: integer, nextAnak: AdrAnak >

type ListPeg: AdrPeg

FirstPeg: ListPeg



KAMUS

type AdrPeg: ... { *type terdefinisi, alamat sebuah elemen list pegawai* }

type AdrAnak: ... { *type terdefinisi, alamat sebuah elemen list anak* }

type Pegawai: < nip: integer, nama: string, jabatan: string,
gajiPokok: real, nextPeg: AdrPeg >

type Anak: < nama: string, tglLahir: integer, nextAnak: AdrAnak,
father: AdrPeg>

type ListPeg: AdrPeg

type ListAnak: AdrAnak

FirstPeg: ListPeg

FirstAnak: ListAnak

Fitur Program

1. Daftar pegawai, dan untuk setiap pegawai harus dibuat juga nama anak-anaknya (jika ada).
2. Daftar anak-anak yang umurnya kurang dari 18 tahun (untuk keperluan tunjangan).
3. Daftar pegawai yang anaknya lebih dari 3 (keperluan KB).
4. Diketahui nama seorang anak, harus dituliskan nama orang tuanya.
5. Mendaftarkan seorang anak yang baru lahir ke dalam list anak, **jika diberikan tanggal lahir dan nama anaknya, dan NIP orang tuanya.**

Alternatif 1: hal 183–187

Alternatif 2: hal 188–191

1. Daftar Nama Pegawai & Anaknya

Alternatif-1:

```
Loop list pegawai
    output nama-pegawai
    Loop list anak dari pegawai
        output nama-anak
```

Alternatif-2:

```
Loop list pegawai
    output nama-pegawai
    Loop list anak
        if father(anak)=pegawai then
            output nama-anak
```

Kedua alternatif:

Perlu penanganan kasus kosong: "List kosong, tidak ada pegawai"

Perlu penanganan kasus kosong: "Pegawai ybs. tidak mempunyai anak"

Alternatif 1 (hal. 184)

```
procedure ListPegLengkap(input FirstPeg: ListPeg)
```

KAMUS LOKAL

PtrPeg: AdrPeg { address untuk traversal, @ sebuah elemen list pegawai }

PtrAnak: AdrAnak { address untuk traversal, @ sebuah elemen list anak }

ALGORITMA

```
{ Traversal pegawai:skema pemrosesan sekuensial dg penanganan kasus kosong
  Untuk setiap pegawai, traversal list anak untuk dituliskan namanya }
```

if (FirstPeg = NIL) then

```
output("List kosong, tidak ada pegawai")
```

```
else { Minimal 1 Pegawai }
```

PtrPeg $\leftarrow$ FirstPeg { First Pegawai }

repeat

```
output(Nama(PtrPeg))
```

{ Traversal Anak }

```
PtrAnak ← FirstAnak(PtrPeg)           { First Anak }
```

if (PtrAnak = NIL) **then**

```
output("Pegawai ybs. tidak mempunyai anak")
```

else

repeat

```
output(Nama(PtrAnak))    { Proses anak }
```

```
PtrAnak ← NextAnak(PtrAnak) { Next Anak }
```

until (PtrAnak = NIL)

PtrPeg ← NextPeg(PtrPeg) { Next Pegawai }

until (PtrPeg = NIL)

Alternatif 2 (hal. 189)

procedure ListPegLengkap(input FirstPeg: ListPeg, input FirstAnak: ListAnak)

KAMUS LOKAL

PtrPeg: AdrPeg { *address untuk traversal, @ sebuah elemen list pegawai* }

PtrAnak: AdrAnak { *address untuk traversal, @ sebuah elemen list anak* }

JmlAnak: integer

ALGORITMA

{ *Traversal pegawai* }

if (FirstPeg = NIL) then output("List kosong, tidak ada pegawai")

else

PtrPeg ← FirstPeg { *First Pegawai* }

repeat

output (Nama(PtrPeg))

{ *Traversal Anak* }

PtrAnak ← FirstAnak { *First Anak* }

JmlAnak ← 0

while (PtrAnak ≠ NIL) do

if (Father(PtrAnak) = PtrPeg) then { *Proses* }

output(Nama(PtrAnak));

JmlAnak ← *JmlAnak*+1

PtrAnak ← NextAnak(PtrAnak) { *Next Anak* }

{ *PtrAnak = NIL* }

if *JmlAnak*=0 then output ("Pegawai tidak/belum mempunyai anak")

PtrPeg ← NextPeg(PtrPeg) { *Next Pegawai* }

until (PtrPeg = NIL)

2. Daftar Anak < 18 tahun (Tunj.)

Alternatif-1:

```
Loop list pegawai
    output nama-pegawai
    Loop list anak dari pegawai
        if umur(anak)<18 then
            output nama-anak
```

Alternatif-2:

```
Loop list anak
    if umur(anak)<18 then
        output nama-anak, nama-father(anak)
Catatan: tidak perlu traversal list pegawai
```

Kedua alternatif:

Perlu penanganan kasus kosong

Alternatif 1 (hal. 185)

procedure ListTunjAnak(input FirstPeg: ListPeg)

{ deklarasi variabel tidak ditulis untuk menghemat tempat }

ALGORITMA

{ Trav.list Pegawai, skema sekuensial dg penanganan kasus kosong.

Untuk setiap pegawai, traversal anaknya }

if (FirstPeg = NIL) then output("List kosong, tidak ada pegawai")

else *{ Minimal ada satu pegawai }*

PtrPeg ← FirstPeg *{ First Pegawai }*

repeat

output(Nama(PtrPeg))

{ Traversal Anak }

PtrAnak ← FirstAnak(PtrPeg) *{ First anak }*

if (PtrAnak = NIL) then

output("Pegawai ybs tidak mempunyai anak")

else *{ Minimal ada 1 anak }*

counter ← 0

repeat

UmurAnak ← Umur(TglLahir(PtrAnak)) *{ Proses }*

if UmurAnak < 18 then

output(Nama(PtrAnak), UmurAnak)

counter ← counter + 1

PtrAnak ← NextAnak(PtrAnak) *{ Next Anak }*

until (PtrAnak=NIL)

if (counter=0) then output("Tidak ada anak pegawai berumur < 18")

PtrPeg ← NextPeg(PtrPeg) *{ Next Pegawai }*

until (PtrPeg=NIL)

Alternatif 2 (hal. 189)

procedure ListTunjAnak (input FirstAnak: ListAnak)

KAMUS LOKAL

PtrAnak: AdrAnak { *address utk traversal, @sbh elemen list anak* }

UmurAnak: integer { *umur anak pegawai* }

function Umur (TglLahir: integer) → integer

{ *Fungsi yg mengirim umur dgn rumus: tgl hari ini dr sistem
dikurangi TglLahir* }

ALGORITMA

{ *Trav.list anak, skema proses sekuensial dg penanganan kasus kosong* }

{ *Untuk setiap anak periksa umurnya* }

if (FirstAnak = NIL) then

output("List Anak kosong, tidak ada anak")

else

 PtrAnak ← FirstAnak { *First Anak* }

repeat

 UmurAnak ← Umur(TglLahir(PtrAnak)) { *Proses* }

if UmurAnak < 18 then

output(Nama(Father(PtrAnak)))

output(Nama(PtrAnak), UmurAnak)

 PtrAnak ← NextAnak(PtrAnak) { *Next Anak* }

until (PtrAnak = NIL)

3. Daftar pegawai dg anak>3 (KB).

Alternatif-1:

```
Loop list pegawai
    Hitung anak pegawai dgn loop list anaknya
    if jumlah-anak>3 then
        output nama-pegawai, status anak>3
```

Alternatif-2 (pola ListPegLengkap):

```
Loop list pegawai
    Hitung anak pegawai dgn loop list anak
    if jumlah-anak>3 then
        output nama-pegawai, status-anak>3
```

Kedua alternatif:

Perlu penanganan kasus kosong

Alternatif 1 (hal. 186)

procedure ListPegNonKB(input FirstPeg: ListPeg)

{ I.S. List FirstPeg terdefinisi, mungkin kosong }

{ F.S. Semua pegawai yg anaknya > 3 orang ditulis informasinya }

KAMUS LOKAL

PtrPeg: AdrPeg { address utk traversal, @ elemen list pegawai }

PtrAnak: AdrAnak { address utk traversal, @ elemen list anak }

JumlahAnak: integer { banyaknya anak pegawai }

ALGORITMA

{ Traversal pegawai }

if (FirstPeg = NIL) then

output("List kosong, tidak ada pegawai")

else { minimal ada satu pegawai }

PtrPeg ← FirstPeg { First-Pegawai }

repeat

{ Traversal Anak }

JumlahAnak ← 0 { Inisialisasi }

PtrAnak ← FirstAnak(PtrPeg) { First Anak }

while (PtrAnak ≠ NIL) do

JumlahAnak ← JumlahAnak + 1 { Proses }

PtrAnak ← NextAnak(PtrAnak) { Next Anak }

if (JumlahAnak > 3) then

output(Nama(PtrPeg), " mempunyai anak > 3")

PtrPeg ← NextPeg(PtrPeg) { Next Pegawai }

until (PtrPeg=NIL)

Alternatif 2 (hal. 190)

procedure ListPegNonKB(input FirstPeg: ListPeg, input FirstAnak: ListAnak)

{ I.S. List First Peg terdefinisi, mungkin kosong }

{ F.S. Semua pegawai yang anaknya > 3 orang ditulis informasinya }

KAMUS LOKAL

PtrPeg: AdrPeg *{ address utk traversal, @ elemen list pegawai }*

PtrAnak: AdrAnak *{ address utk traversal, @ elemen list anak }*

JumlahAnak: integer *{ banyaknya anak pegawai }*

ALGORITMA

{ Traversal list Pegawai: skema sekuensial dg penanganan kasus kosong }

{ Untuk setiap pegawai, traversal list anak utk mencacah jumlah anaknya.

Jika jumlah anak > tiga maka nama pegawai ditulis }

{ Traversal pegawai }

if (FirstPeg = NIL) then output("List pegawai kosong")

else

PtrPeg ← FirstPeg *{ First Pegawai }*

repeat *{ Proses }*

 JumlahAnak ← 0

{ Traversal Anak }

 PtrAnak ← FirstAnak *{ First Anak }*

while (PtrAnak ≠ NIL) do

if (Father(PtrAnak) = PtrPeg) then *{ Proses Anak }*

 JumlahAnak ← JumlahAnak + 1

 PtrAnak ← NextAnak(PtrAnak) *{ Next Anak }*

{ PtrAnak = NIL }

if (JumlahAnak > 3) then output(Nama(PtrPeg), " mempunyai anak lebih dari 3")

 PtrPeg ← NextPeg(PtrPeg) *{ Next Pegawai }*

until (PtrPeg = NIL)

{ semua elemen list pegawai selesai diproses }

4. Search Nama Orang Tua dari Anak

Alternatif-1:

```
Loop list pegawai
  Search>NamaAnak dg loop list anak dr pegawai
    if>NamaAnak ketemu then
      output nama-pegawai
    keluar dr loop anak dan loop pegawai
```

Alternatif-2:

```
Search>NamaAnak dgn loop list anak
  if>NamaAnak ketemu then
    output nama-pegawai
  keluar dr loop list anak
```

Alternatif 1 (hal. 186)

```
procedure OrTuAnak(input FirstPeg: ListPeg, input>NamaAnak: string)
{ I.S. List Pegawai terdefinisi }
{ F.S. Jika ada anak yg bernama sesuai dg>NamaAnak, nama Pegawai ditulis.
  Jika tidak ada>NamaAnak, tidak menuliskan apa-apa }
```

KAMUS LOKAL

```
PtrPeg: AdrPeg    { address utk traversal, @ elemen list pegawai }
PtrAnak: AdrAnak  { address utk traversal, @ elemen list anak }
Found: boolean    { hasil pencarian orangtua anak }
```

ALGORITMA

```
{ Search }
Found ← false
PtrPeg ← FirstPeg
while (PtrPeg ≠ NIL) and (not Found) do
  { Search anak dengan>NamaAnak yang diberikan pada list anak }
  PtrAnak ← FirstAnak(PtrPeg)
  while (PtrAnak ≠ NIL) and (not Found) do
    if (Nama(PtrAnak) =>NamaAnak) then Found ← true
    else PtrAnak ← NextAnak(PtrAnak)
  { PtrAnak = NIL or Found }
  if (not Found) then { explore pegawai yg berikutnya }
    PtrPeg ← NextPeg(PtrPeg)
  { PtrPeg = NIL or Found }
if (Found) then output(Nama(PtrPeg))
```

Alternatif 2 (hal. 190)

```
procedure OrTuAnak(input FirstAnak: ListAnak, input>NamaAnak: string)
{ Alternatif Kedua }
{ I.S. List Pegawai terdefinisi }
{ F.S. Jika ada anak yg bernama>NamaAnak, nama Pegawai ditulis.
  Jika tidak ada>NamaAnak, tidak menuliskan apa-apa. }
```

KAMUS LOKAL

```
PtrPeg: AdrPeg    { address utk traversal, @ elemen list pegawai }
PtrAnak: AdrAnak { address utk traversal, @ elemen list anak }
Found: boolean   { hasil pencarian orangtua anak }
```

ALGORITMA

```
{ Search pada List Anak berdasarkan nama. Jika ketemu, akses Bapaknya }
PtrAnak ← FirstAnak; Found ← false
while (PtrAnak ≠ NIL) and (not Found) do
  if (Nama(PtrAnak) =>NamaAnak) then
    Found ← true
  else
    PtrAnak ← NextAnak(PtrAnak)
{ PtrAnak = NIL or Found }
if (Found) then
  output(Nama(Father(PtrAnak)))
```

5. Mendaftarkan anak yang baru lahir

Alternatif-1:

Search elemen dgn NIP_{Peg} pd list pegawai

If ketemu then

 Insert first data-anak pd list anaknya

Alternatif-2:

Search elemen dgn NIP_{Peg} pd list pegawai

If ketemu then

 Insert first data-anak pada list anak

Alternatif 1 (hal. 187)

```
procedure AddAnak (input/output FirstPeg: ListPeg, input NIPPeg: string,  
                    input>NamaAnak: string, input TglLahirAnak: integer)  
{ Mendaftar seorang anak yang baru lahir, insert selalu pada awal list }  
{ I.S. List Pegawai terdefinisi }  
{ F.S. Jika pegawai dgn NIP=NIPPeg ada, alokasi anak.  
    Jika berhasil insert seorang anak sebagai elemen pertama list anak.  
    Jika alokasi gagal atau NIPPeg tidak ada, hanya menulis pesan }
```

KAMUS LOKAL

```
PtrPeg: AdrPeg    { address utk traversal, @ elemen list pegawai }  
PtrAnak: AdrAnak { address utk traversal, @ elemen list anak }  
FoundNIP: boolean { hasil pencarian NIP pegawai sebelum insert anak }
```

ALGORITMA

```
{ Search Pegawai }  
FoundNIP ← false  
PtrPeg ← FirstPeg  
while (PtrPeg ≠ NIL) and (not FoundNIP) do  
    { search Pegawai dengan NIP yang diberikan }  
    if (NIP(PtrPeg) = NIPPeg) then  
        FoundNIP ← true  
    else  
        PtrPeg ← NextPeg(PtrPeg)  
...  

```

Alternatif 1 (lanjutan)

```
...  
{ Akhir search pegawai: PtrPeg=NIL or FoundNIP }  
if (FoundNIP) then      { add anak }  
    PtrAnak ← Alokasi>NamaAnak , TglLahirAnak)  
    if (PtrAnak ≠ NIL) then  
        NextAnak(PtrAnak) ← FirstAnak(PtrPeg)  
        FirstAnak(PtrPeg) ← PtrAnak  
    else { Alokasi gagal, tidak insert, hanya pesan }  
        output("Alokasi gagal")  
else { NIPPeg tidak ada, error }  
    output("Pegawai tidak ada dalam list")
```

Alternatif 2 (hal. 191)

```
procedure AddAnak (input/output FirstPeg:ListPeg, input NIPPeg:integer,  
                    input/output FirstAnak:ListAnak,  
                    input NamaAnak:string, input TglLahirAnak:integer)  
{ Mendaftar seorang anak yang baru lahir,  
  Cari Pegawai, insert anak pada list anak selalu pada awal list }  
{ I.S. List Pegawai terdefinisi }  
{ F.S. List FirstPeg terdefinisi.  
  Jika pegawai dengan NIP=NIPPeg ada, alokasi anak.  
  Jika alokasi berhasil, insert seorg anak sbg elemen pertama list  
  anak, tentukan Bapak.  
  Jika alokasi gagal atau NIPPeg tidak ada, hanya menulis pesan. }
```

KAMUS LOKAL

```
PtrPeg: AdrPeg    { address utk traversal, @ elemen list pegawai }  
PtrAnak: AdrAnak  { address utk traversal, @ elemen list anak }  
FoundNIP: boolean { hasil pencarian NIP pegawai sbkm insert anak }
```

...

Alternatif 2 (lanjutan)

ALGORITMA

```
{ Search Pegawai dgn NIP yg diberikan: skema search dgn boolean }
FoundNIP ← false
PtrPeg ← FirstPeg
while (PtrPeg ≠ NIL) and (not FoundNIP) do
    if (NIP(PtrPeg) = NIPPeg) then
        FoundNIP ← true
    else
        PtrPeg ← NextPeg(PtrPeg)
{ PtrPeg = NIL or FoundNIP }
{ Akhir search pegawai: PtrPeg=NIL or FoundNIP }
if (FoundNIP) then    { Insert anak }
    PtrAnak ← Alokasi>NamaAnak, TglLahir)
    if (PtrAnak ≠ NIL) then
        Father(PtrAnak) ← Ptrpeg { Tentukan Bapaknya }
        { Insert Anak, kasus kosong ditangani dg cara yg sama dengan
          kasus tidak kosong }
        NextAnak(PtrAnak) ← FirstAnak
        FirstAnak ← PtrAnak
    else { Alokasi gagal:tidak melakukan apa-apa, hanya pesan }
        output("Alokasi gagal")
else { NIPPeg tidak ada, error }
    output("Pegawai tidak ada dalam list")
```

Studi Kasus: Representasi Relasi M-N

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Deskripsi Persoalan

Sistem harus mengelola sekumpulan dosen dan sekumpulan mata kuliah serta data pengajaran (mata kuliah MK diajar oleh dosen D). Seorang **dosen dapat mengajar lebih dari satu mata kuliah** dan **sebuah mata kuliah dapat diajar oleh lebih dari satu dosen**.

→ relasi antara Dosen dan Mata kuliah adalah $M-N$

Jika informasi tersebut harus direpresentasikan dalam struktur data internal, akan terdapat list dari dosen dan juga list dari mata kuliah.

Masalahnya adalah bagaimana merepresentasikan daftar pengajaran (relasi antara dosen dan mata kuliah)?

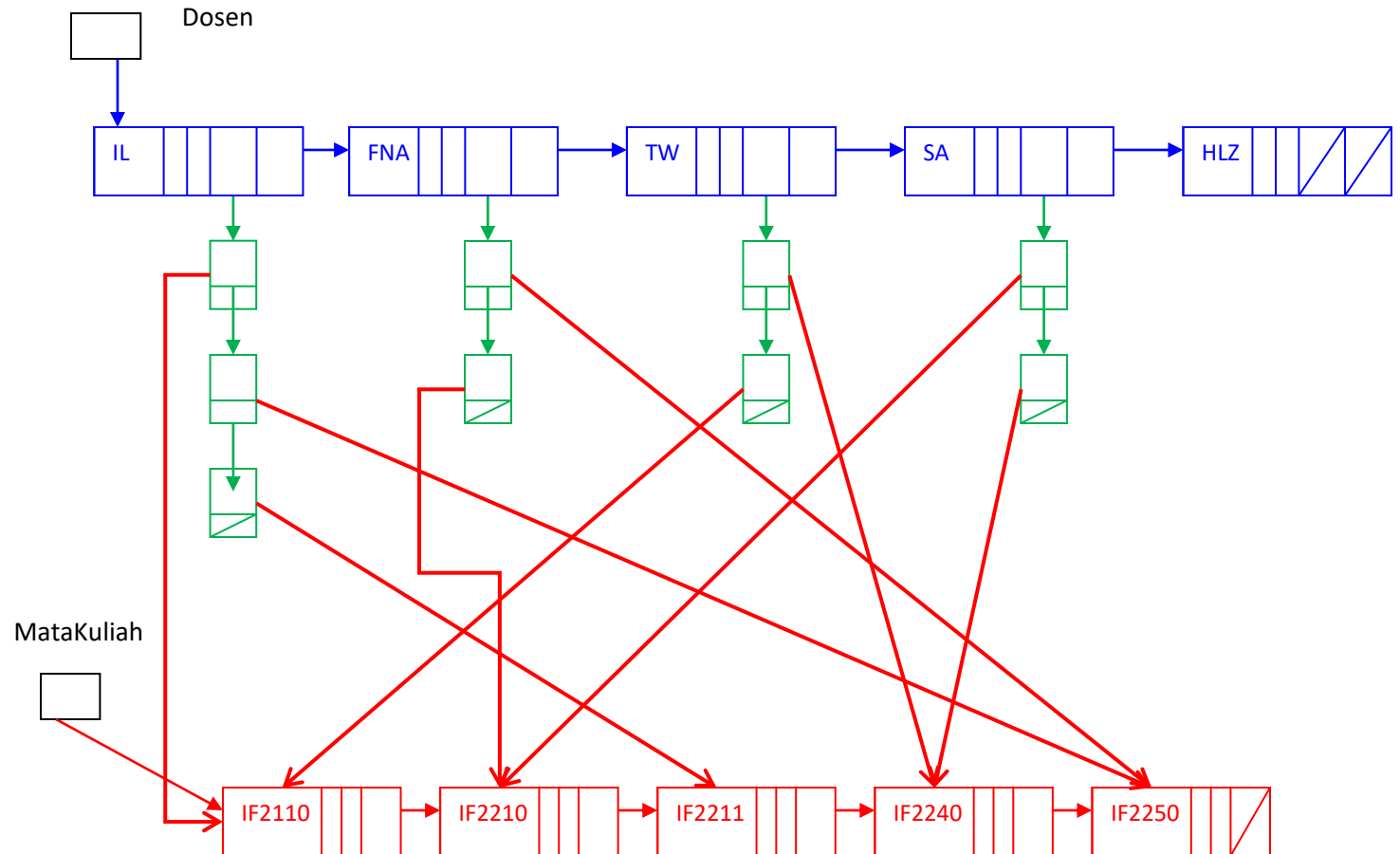
Alternatif Representasi Relasi M-N

1. Relasi “Mengajar”
2. Relasi “Diajar Oleh”
3. Relasi hubungan “Dosen_MataKuliah”

Alternatif 1: Relasi sebagai “Mengajar”

Relasi “Mengajar”

Untuk merepresentasi relasi dari sudut pandang setiap Dosen: setiap Dosen mempunyai list MataKuliah yang diajarnya. Hubungan ini $1-N$ dipandang dari list Dosen. Dengan representasi ini, setiap elemen list Dosen akan mempunyai list of MataKuliah yang diajarnya.



Alternatif 2: Relasi sebagai “Diajar Oleh”

Relasi “Diajar oleh”

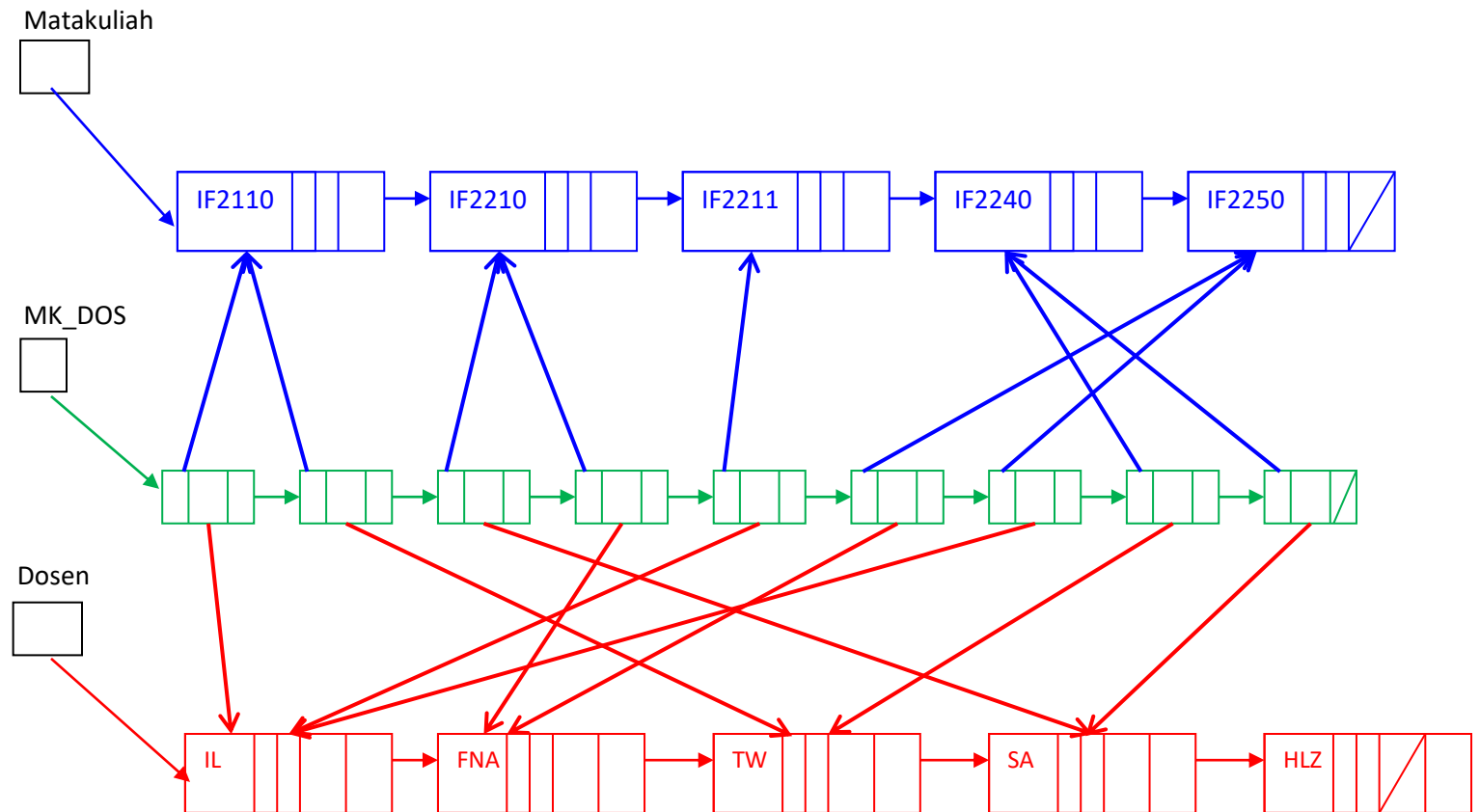
Untuk merepresentasi relasi dari sudut pandang setiap MataKuliah: sebuah MataKuliah diajarkan oleh siapa saja. Hubungan ini adalah hubungan $1-N$ dipandang dari list MataKuliah. Dengan representasi ini, setiap elemen list MataKuliah akan mempunyai list of Dosen pengajarnya

Menyerupai alternatif 1, hanya saja yang menjadi parent dari relasi adalah relasi Mata kuliah.

Alternatif 3: Relasi sebagai list terpisah

Relasi hubungan “Dosen_MataKuliah”

Yaitu untuk merepresentasi setiap relasi MataKuliah dan Dosen: setiap elemen relasi $\langle \text{Dosen}, \text{MataKuliah} \rangle$ adalah unik, maka relasi hubungan ini dikelola sebagai list yang terpisah.



Perbandingan Ketiga Alternatif

Buatlah studi perbandingan terhadap ketiga representasi tersebut terutama dari sisi efisiensi proses, jika misalnya sistem harus mampu menampilkan:

- Daftar mata kuliah yang diajar oleh dosen X
- Daftar dosen yang mengajar mata kuliah Y
- Jumlah mata kuliah yang diajar oleh setiap dosen

Penambahan Relasi Baru

Procedure **AddRel** menerima $\langle D, MK \rangle$ dan menambahkan sebuah relasi Dosen MataKuliah, dengan ketentuan:

- jika D belum ada di list Dosen maka ditambahkan lebih dulu sebagai elemen list. Demikian pula jika MK belum ada, maka sebelum menambahkan relasi, elemen list mata kuliah ditambahkan lebih dulu.
- jika D dan MK sudah ada pada list Dosen dan Matakuliah, maka $\langle D, MK \rangle$ harus belum muncul dalam list relasi (harus unik).

Studi Kasus: Pengembangan Representasi

Coba kembangkan masing-masing alternatif untuk kasus berikut ini:

- Sebuah relasi dapat terjadi antara elemen list yang sama. Misalnya mata kuliah berelasi dengan mata kuliah lain untuk menyatakan daftar mata kuliah yang menjadi prerequisite dari suatu mata kuliah.
- Sebuah himpunan objek dapat terkait dengan lebih dari satu relasi. Misalnya mata kuliah terlibat dalam relasi pengajaran dan prerequisite.